

# RL Learning Platform

DigitalOcean Deployment Guide

## Step-by-Step Production Setup

2026-05-22

|                                          |                                     |                                    |                         |
|------------------------------------------|-------------------------------------|------------------------------------|-------------------------|
| <b>Platform</b><br>Flutter Web + FastAPI | <b>Host</b><br>DigitalOcean Droplet | <b>Container</b><br>Docker Compose | <b>Auth</b><br>Firebase |
|------------------------------------------|-------------------------------------|------------------------------------|-------------------------|

## Table of Contents

---

- 1. Overview**
- 2. Prerequisites**
- 3. Step 1 — Create the DigitalOcean Droplet**
- 4. Step 2 — Bootstrap the Droplet**
- 5. Step 3 — Upload Secrets**
- 6. Step 4 — Configure Environment Variables**
- 7. Step 5 — SSL with Let's Encrypt**
- 8. Step 6 — Build and Start the Platform**
- 9. Step 7 — Bootstrap the Admin Account**
- 10. Step 8 — Configure CI/CD with GitHub Actions**
- 11. Ongoing Operations**
- 12. Troubleshooting**
- 13. Files Created in This Deployment**

## 1. Overview

The **RL Learning Platform** is a research and teaching web application that enables students to learn Reinforcement Learning through an interactive browser-based IDE with animated replays, an AI coaching assistant (Study Buddy), and quiz-based assessment.

The platform is composed of four Docker containers:

- **API Gateway** (port 8000) — Public HTTP entry point; routes Flutter requests; proxies to internal services
- **Execution Worker** (port 8100) — Code validation, execution sandbox, Manim rendering
- **User / Evaluation Service** (port 8200) — Student progress, quiz engine, N-gain calculation
- **Frontend / Nginx** (port 80/443) — Serves compiled Flutter web app; reverse-proxies API calls to gateway

### Architecture

```
Internet
| HTTPS :443 / HTTP :80
v
[Nginx / Frontend container]
|-- Static files --> Flutter web app (compiled)
+-- API proxy --> [Gateway :8000]
|-- --> [Execution Worker :8100]
+-- --> [User-Eval Service :8200]
```

### Recommended Droplet Size

| Tier        | Spec              | Monthly Cost | Use Case                  |
|-------------|-------------------|--------------|---------------------------|
| Minimum     | 2 vCPU / 2 GB RAM | \$12 / mo    | Light testing, no Manim   |
| Recommended | 2 vCPU / 4 GB RAM | \$24 / mo    | Full platform with Manim  |
| Comfortable | 4 vCPU / 8 GB RAM | \$48 / mo    | Multiple concurrent users |

**Note:** Manim (the RL animation renderer) is CPU and memory intensive. A 2 GB Droplet may swap during rendering — 4 GB is strongly recommended.

## 2. Prerequisites

---

Before starting, ensure you have the following ready:

| Item                   | Details                                                            |
|------------------------|--------------------------------------------------------------------|
| DigitalOcean account   | Account with credits; a payment method on file                     |
| Domain name (optional) | A-record pointing to Droplet IP — required for HTTPS               |
| Firebase project       | 'rlplat' project; Authentication enabled; service account JSON key |
| Admin Firebase UID     | From Firebase Console → Authentication → Users → copy UID          |
| GitHub repository      | Codebase pushed; Actions enabled; Packages (ghcr.io) enabled       |
| SSH key pair           | Local key for Droplet access; separate deploy key for CI/CD        |
| Git (local machine)    | To commit and push deployment files                                |

### Firebase Setup Checklist

- Firebase Console → Authentication → Sign-in method → enable **Email/Password**
- Firebase Console → Authentication → Sign-in method → enable **Google**
- Firebase Console → Project Settings → Service Accounts → **Generate new private key** → save as *firebase-admin-key.json*
- Note the Firebase UID of your admin account (appears after first sign-in)

## 3. Step 1 — Create the DigitalOcean Droplet

- 1. Log in to DigitalOcean → click **Create** → **Droplets**
- 2. OS: **Ubuntu 22.04 LTS** (x64)
- 3. Plan: **Basic** → **Regular CPU** → **4 GB / 2 vCPU**
- 4. Region: choose the datacenter closest to your users
- 5. Authentication: **SSH Key** → paste your public key
- 6. Click **Create Droplet** — note the public **IP address**

### DNS Configuration (for custom domain)

Add an **A record** in your DNS provider pointing your domain to the Droplet IP. DNS propagation can take up to 24 hours — use the IP address directly until it resolves.

```
# Example DNS record
Type Name Value
A your-domain.com <Droplet-IP>
A www <Droplet-IP> (optional)
```

## 4. Step 2 — Bootstrap the Droplet

SSH into the fresh Droplet, clone the repository, and run the provided bootstrap script. The script is idempotent — safe to re-run.

```
# 1. SSH in
ssh ubuntu@<droplet-ip>

# 2. Clone the repo
git clone https://github.com/YOUR_USERNAME/YOUR_REPO.git /home/ubuntu/rl-platform
cd /home/ubuntu/rl-platform

# 3. Run bootstrap (installs Docker, UFW, data directories)
sudo bash deploy/setup-droplet.sh --user ubuntu

# 4. Log out and back in (docker group change takes effect)
exit
ssh ubuntu@<droplet-ip>
```

The bootstrap script performs these 8 steps:

| # | Action          | Detail                                                       |
|---|-----------------|--------------------------------------------------------------|
| 1 | System packages | Updates apt; installs curl, gnupg, ufw, git, ca-certificates |

| # | Action           | Detail                                                                          |
|---|------------------|---------------------------------------------------------------------------------|
| 2 | Docker Engine    | Adds Docker's official apt repo (not snap); installs docker-ce + compose-plugin |
| 3 | Docker service   | systemctl enable --now docker                                                   |
| 4 | Docker group     | Adds ubuntu user — log out/in required                                          |
| 5 | Data directories | Creates /srv/rl-platform/{data,animations,concept-videos,secrets,letsencrypt}   |
| 6 | Ownership        | chown -R ubuntu:ubuntu /srv/rl-platform                                         |
| 7 | UFW firewall     | Allow SSH (22), HTTP (80), HTTPS (443) — deny everything else                   |
| 8 | Certbot          | Installs certbot for Let's Encrypt SSL issuance                                 |

## 5. Step 3 — Upload Secrets

Upload the Firebase service account key and any pre-rendered concept videos from your local machine to the Droplet.

```
# Upload Firebase service account key
scp /path/to/firebase-admin-key.json \
ubuntu@<droplet-ip>:/srv/rl-platform/secrets/firebase-admin-key.json

# (Optional) Upload pre-rendered concept videos
scp -r /path/to/concept_videos/* \
ubuntu@<droplet-ip>:/srv/rl-platform/concept-videos/
```

**Note:** The Firebase key is referenced by the Docker secrets mechanism in `docker-compose.prod.yml`. The path `/srv/rl-platform/secrets/` is owned by `ubuntu` and is not world-readable after the bootstrap script runs.

## 6. Step 4 — Configure Environment Variables

Copy the production environment template and fill in all required values. Generate secrets with the Python one-liner shown below — do not reuse secrets across environments.

```
cd /home/ubuntu/rl-platform
cp deploy/prod.env.template .env
nano .env

# Generate a secret token (run 3 times – one per secret variable):
python3 -c "import secrets; print(secrets.token_urlsafe(32))"
```

### Required Environment Variables

| Variable                            | How to Obtain                                                              |
|-------------------------------------|----------------------------------------------------------------------------|
| RL_IDE_INTERNAL_TOKEN               | <code>python3 -c "import secrets; print(secrets.token_urlsafe(32))"</code> |
| RL_IDE_AUTH_TOKEN_SECRET            | Same command as above                                                      |
| RL_IDE_SHELL_TOKEN_SECRET           | Same command as above                                                      |
| RL_IDE_FIREBASE_CREDENTIALS_FILE    | <code>/srv/rl-platform/secrets/firebase-admin-key.json</code>              |
| RL_IDE_BOOTSTRAP_ADMIN_FIREBASE_UID | Firebase Console > Authentication > Users > copy UID                       |
| RL_IDE_ALLOWED_ORIGINS              | <code>https://your-domain.com</code> (comma-sep for multiple)              |

| Variable              | How to Obtain                                   |
|-----------------------|-------------------------------------------------|
| RL_IDE_IMAGE          | ghcr.io/YOUR_USERNAME/YOUR_REPO-backend:latest  |
| RL_IDE_FRONTEND_IMAGE | ghcr.io/YOUR_USERNAME/YOUR_REPO-frontend:latest |

Production Security Flags

RL\_IDE\_OPEN\_PROVISIONING=0 # Disables auto-approval for new accounts  
RL\_IDE\_ALLOW\_LEGACY\_PASSWORD\_SIGN\_IN=0 # Disables legacy auth path



## 7. Step 5 — SSL with Let's Encrypt

---

Issue a TLS certificate before starting the stack so the standalone certbot challenge can bind to port 80 uncontested.

```
# Issue certificate (port 80 must be free)
sudo certbot certonly --standalone -d your-domain.com

# Certificates are stored at:
# /etc/letsencrypt/live/your-domain.com/fullchain.pem
# /etc/letsencrypt/live/your-domain.com/privkey.pem

# Enable auto-renewal via cron:
echo "0 12 * * * certbot renew --quiet" | sudo crontab -
```

### Enable SSL in Docker Compose

To use HTTPS, add a volume mount for the certificates to the frontend service in `docker-compose.prod.yml` and switch the Nginx config to the SSL variant:

```
# In docker-compose.prod.yml, under the frontend service:
volumes:
- /etc/letsencrypt:/etc/letsencrypt:ro

# In Dockerfile.frontend, change the nginx config line to:
COPY nginx/nginx-ssl.conf /etc/nginx/conf.d/default.conf

# Replace DOMAIN in nginx-ssl.conf with your actual domain,
# then rebuild the frontend image:
docker compose -f docker-compose.prod.yml build frontend
docker compose -f docker-compose.prod.yml up -d frontend
```

**Note:** For the first deployment, HTTP-only is acceptable for smoke-testing. Add HTTPS once the stack is confirmed healthy.

## 8. Step 6 — Build and Start the Platform

---

```
# Log in to GitHub Container Registry
echo YOUR_GITHUB_PAT | docker login ghcr.io -u YOUR_GITHUB_USERNAME --password-stdin

# Pull pre-built images (after CI/CD has run at least once)
docker compose -f docker-compose.prod.yml --env-file .env pull

# First-time start (builds locally if images not yet available):
docker compose -f docker-compose.prod.yml --env-file .env up -d --build

# Monitor startup logs:
docker compose -f docker-compose.prod.yml logs -f

# Confirm all 4 services are healthy:
docker compose -f docker-compose.prod.yml ps

# Smoke test:
curl http://localhost/health
```

## Expected Healthy State

| Service          | Port | Health Endpoint            | Expected Status |
|------------------|------|----------------------------|-----------------|
| frontend         | 80   | / (Nginx)                  | healthy         |
| gateway          | 8000 | /health                    | healthy         |
| execution-worker | 8100 | /internal/workspace/health | healthy         |
| user-evaluation  | 8200 | / (root)                   | healthy         |

## 9. Step 7 — Bootstrap the Admin Account

On first startup the gateway automatically creates an admin record for the Firebase UID in `RL_IDE_BOOTSTRAP_ADMIN_FIREBASE_UID`. Sign in as that user to verify admin access.

- Open the platform in a browser: **`http://your-domain.com`**
- Sign in with the Firebase account whose UID was set in the env file
- Navigate to **Admin Dashboard** — confirm student management and analytics are accessible
- Create a test student account to verify the full registration flow

## 10. Step 8 — Configure CI/CD with GitHub Actions

The pipeline at `.github/workflows/deploy.yml` runs automatically on every push to `main`. It has three jobs:

```
Push to main
|
v
[Job 1: test]
|-- pytest (backend) -- Python 3.11
+-- flutter analyze/test -- Flutter stable
|
v (on main only)
[Job 2: build-and-push]
|-- Build Dockerfile.cloud --> ghcr.io/<repo>-backend:latest
+-- Build Dockerfile.frontend --> ghcr.io/<repo>-frontend:latest
|
v
[Job 3: deploy]
|-- SCP docker-compose.prod.yml to Droplet
+-- SSH: docker compose pull && up -d --remove-orphans
```

### Required GitHub Secrets

Go to: GitHub repo → Settings → Secrets and variables → Actions → New repository secret

| Secret Name | Value                                                                     |
|-------------|---------------------------------------------------------------------------|
| DO_HOST     | Droplet IP address or hostname                                            |
| DO_USER     | ubuntu (or root)                                                          |
| DO_SSH_KEY  | Full PEM private key — paste entire content including header/footer lines |

| Secret Name    | Value                    |
|----------------|--------------------------|
| DO_DEPLOY_PATH | /home/ubuntu/rl-platform |

## Generate a Dedicated Deploy SSH Key

```
# On your local machine:
ssh-keygen -t ed25519 -C 'github-actions-deploy' -f ~/.ssh/rl_platform_deploy

# Authorise the key on the Droplet:
ssh-copy-id -i ~/.ssh/rl_platform_deploy.pub ubuntu@<droplet-ip>

# Paste the PRIVATE key into the DO_SSH_KEY GitHub secret:
cat ~/.ssh/rl_platform_deploy
```

## Trigger the First Deploy

```
git add .
git commit -m "Add DigitalOcean deployment configuration"
git push origin main

# Monitor at: GitHub repo → Actions tab
```

## 11. Ongoing Operations

---

### View logs

```
docker compose -f docker-compose.prod.yml logs -f gateway
docker compose -f docker-compose.prod.yml logs -f execution-worker
```

### Restart a service

```
docker compose -f docker-compose.prod.yml restart frontend
```

### Manual update

```
cd /home/ubuntu/rl-platform
docker compose -f docker-compose.prod.yml pull
docker compose -f docker-compose.prod.yml up -d --remove-orphans
docker image prune -f
```

### Database backup

```
mkdir -p ~/backups
cp /srv/rl-platform/data/rl_learning_platform.db \
~/backups/$(date +%Y%m%d)_rl_platform.db
```

### Disk cleanup

```
docker image prune -af
docker volume prune -f
```

### SSL renewal check

```
sudo certbot renew --dry-run
```

## 12. Troubleshooting

| Symptom                             | Likely Cause                                                     | Fix                                                                                                                                 |
|-------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Gateway unhealthy                   | Missing env var or bad Firebase JSON                             | <code>docker logs rl-platform-prod-gateway-1</code> – check for <code>ValueError</code> or <code>credentials error</code>           |
| Execution worker won't start        | Docker socket not mounted or user not in docker group            | Verify <code>/var/run/docker.sock</code> volume mount; log out and back in after bootstrap                                          |
| Flutter app: 'Backend disconnected' | Nginx not proxying to gateway, or gateway unhealthy              | <code>docker compose ps</code> – confirm gateway shows healthy; check <code>nginx.conf</code> proxy rules                           |
| Firebase auth errors                | Wrong credentials path or malformed JSON                         | Verify <code>RL_IDE_FIREBASE_CREDENTIALS_FILE</code> path; validate JSON: <code>python3 -m json.tool firebase-admin-key.json</code> |
| Port 80 already in use              | Another process (e.g. Apache) on port 80                         | <code>sudo lsof -i :80</code> — kill or disable the conflicting process                                                             |
| Out of disk space                   | Accumulated Docker images/volumes                                | <code>docker image prune -af</code> && <code>docker volume prune -f</code>                                                          |
| CI/CD deploy job fails on SSH       | Wrong key, wrong user, or IP not in <code>known_hosts</code>     | Test manually: <code>ssh -i ~/.ssh/rl_platform_deploy ubuntu@</code>                                                                |
| Images not found on ghcr.io         | build-and-push job not run yet, or package visibility is private | Go to GitHub repo Packages → set visibility to Public, or pass <code>GITHUB_TOKEN</code> in compose pull                            |

## 13. Files Created in This Deployment Preparation

| File                                      | Purpose                                                                      |
|-------------------------------------------|------------------------------------------------------------------------------|
| <code>Dockerfile.frontend</code>          | Multi-stage build: Flutter web compile (canvaskit) + Nginx serving           |
| <code>nginx/nginx.conf</code>             | Nginx config: static Flutter files + API reverse proxy + WebSocket support   |
| <code>nginx/nginx-ssl.conf</code>         | HTTPS variant with Let's Encrypt certificate paths                           |
| <code>docker-compose.prod.yml</code>      | Updated: adds frontend/nginx service to the existing 3-service backend stack |
| <code>.github/workflows/deploy.yml</code> | CI/CD: test (pytest + flutter) → build/push Docker images → SSH deploy       |
| <code>deploy/setup-droplet.sh</code>      | Idempotent Ubuntu 22.04 bootstrap: Docker, UFW, directories, Certbot         |
| <code>deploy/prod.env.template</code>     | Production environment variables template with inline documentation          |

| File             | Purpose                                   |
|------------------|-------------------------------------------|
| deploy/README.md | Quick-reference for the deploy/ directory |

End of Deployment Guide